

ReBAC V3.2 — Language Evolution

(Imports, Modules, Namespaces, Inheritance)

Branch: `ReBAC-V3.2` | **Builds on:** `V3.1 Zanzibar Relations`

The Problem V3.1 Left Unsolved

In a real hospital system, the authorization schema grows to thousands of lines — all in one `hospital.auth` file. This is unmaintainable:

- Two teams cannot independently edit the schema without merge conflicts
- Shared concepts (`admin_permission`) must be copy-pasted across resources
- No way to group related resources (billing, pharmacy, clinical) separately

V3.2 solves this with **software engineering concepts applied to the DSL itself** — imports, modules, namespaces, and resource inheritance.

V3.2 Language Features

1. Imports

```
# hospital.auth
import "patient.auth"
import "ward.auth"
import "billing.auth"
import "pharmacy.auth"
```

The **ImportResolver** resolves import paths, reads each file, and merges the compiled ASTs before the semantic analysis pass. This mirrors TypeScript's `import / /// <reference>` system.

Compiler pipeline change:

```
Source → ImportResolver (new) → Lexer → Parser → SemanticAnalyzer →  
...
```

Circular import detection uses the same topological DFS as V3.1's permission cycle detection.

2. Modules

```
module billing {  
  resource invoice {  
    relation owner : user  
    permission view = owner  
  }  
}
```

Modules create a logical grouping scope. Resources inside modules are referenced as `billing::invoice` — the double-colon `::` is the namespace separator, matching C++ and Rust conventions.

3. Namespaces

```
namespace clinical {  
  resource patient { ... }  
  resource ward { ... }  
}  
  
namespace billing {  
  resource invoice { ... }  
  resource payment { ... }  
}
```

Namespaces provide **compile-time isolation** — a resource in `billing` cannot accidentally reference a relation in `clinical` without explicit qualification.

4. Resource Inheritance

```
# base_resource.auth  
resource BaseDocument {  
  relation owner : user
```

```

    relation viewer : user

    permission view = viewer OR owner
    permission delete = owner
  }

# patient.auth
resource patient extends BaseDocument {
  relation doctor_of : user
  # Inherits: view, delete from BaseDocument
  # Adds new permission:
  permission discharge = doctor_of
}

```

The **InheritanceResolver** flattens the inheritance hierarchy before type checking, copying relations and permissions from parent to child, then allowing the child to override or extend them.

Single inheritance is enforced — no diamond problem.

Critical Review

✓ What V3.2 Gets Right

1. **Import resolution** is the correct first step for modular authorization schemas. SpiceDB uses `.zaml` schema files that can import other schema files; OpenFGA uses a JSON schema but recommends splitting across files per bounded context.
2. **Namespaces** correctly solve the naming collision problem in large schemas.
3. **Resource inheritance** is a genuine DRY improvement for shared permission patterns.

⚠ What V3.2 Is Still Missing

Missing: Circular import detection reporting The circular import detection catches cycles but the error message doesn't identify *which* files form the cycle.

Missing: Traits / Mixins Single inheritance is limiting. A `patient` may want to inherit from both `ClinicalResource` and `BillableResource`. Traits (Go interfaces) or Mixins (Scala traits) allow composition without multiple inheritance complexity.

Missing: Schema validation across imports The `SemanticAnalyzer` validates each file in isolation during import resolution. Cross-file type checking (does `patient.auth`'s reference to `user` match the `user` declared in `base.auth`?) is not implemented.

Missing: Module versioning No `module billing @ v2.0` syntax. If a module changes its relations, consumers get no compile-time warning.

✗ Overengineered: Modules + Namespaces

Both modules and namespaces provide logical grouping. Having both creates confusion:

- When should you use a `module` ?
- When should you use a `namespace` ?

Most real systems (SpiceDB, OpenFGA, Ory Keto) use a **single** namespace concept with hierarchical naming (e.g., `clinical/patient` , `billing/invoice`). V3.2 introduces two separate scoping mechanisms that partially overlap.

Recommendation: Consolidate into a single `namespace` concept. Modules can be a file-level concept only.

Comparison: Schema Modularity in Real Systems

System	Modularity	Import	Namespaces
ADSL V3.2	✓ imports + modules + namespaces	✓	✓
SpiceDB	Partial — single schema file	✗	✓ (prefix-based)
OpenFGA	JSON schema, no imports	✗	✓ type names
OPA/Rego	✓ full package imports	✓	✓ package names
AWS Cedar	✓ policy sets	✓	✓ namespaces

ADSL V3.2 is **more modular than SpiceDB or OpenFGA** — and nearly as modular as OPA/Rego, which is a genuine strength.

Summary

V3.2 transforms ADSL from a single-file DSL into a **fully modular authorization policy language** with import resolution, modules, namespaces, and resource inheritance. These are genuine software engineering improvements that mirror how production systems like OPA (Rego packages) and AWS Cedar (policy sets) scale authorization schemas across large teams. The main gap is the overlap between modules and namespaces — consolidating to a single scoping mechanism would improve clarity.